

```
33 self.logdupes = True
34 self.debug = debug
35 self.logger = logging.getLogger(__name__)
36
37 if path:
38     self.file = open(os.path.join(path, "runner.log"), "a")
39     self.file.seek(0)
40     self.fingerprints.update({request.fingerprint()})
```

AGENT-RUNNER

Runner Stack

Prompt-runner, methodology-runner, and backlog-runner

Prompt modules

Methodology phases

Parallel backlog delivery

MENTAL MODEL

Three Layers, Three Jobs

Each runner owns a different unit of work. The stack is useful because those ownership boundaries stay clean.

lowest layer

prompt-runner

Executes a markdown prompt workflow as generator, deterministic validation, judge, and revision loops.

Can this prompt module produce a passing artifact chain?

single change

methodology-runner

Sequences the checked-in methodology phase prompt modules over one application worktree.

Can one request become a verified integrated change?

queue layer

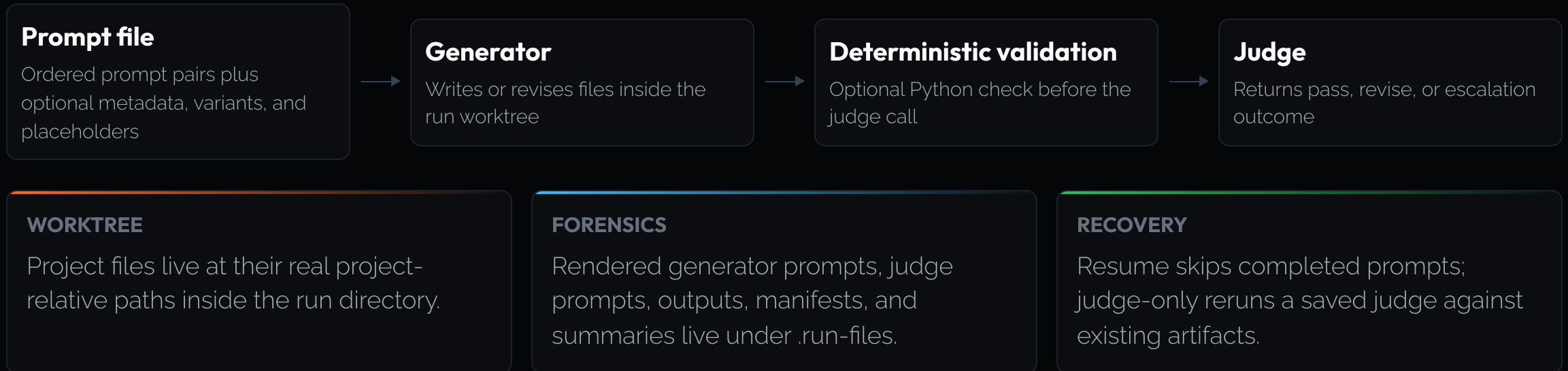
backlog-runner

Scans backlog folders, claims items, launches isolated methodology workers, and serializes final merges.

Can many items move safely through the pipeline?

The Smallest Execution Unit

Prompt-runner is content-agnostic. It cares about markdown structure, files, backend calls, verdicts, and durable traces.



The Markdown Contract

A prompt module is a reusable workflow file. Prompt-runner supplies runtime paths, state, and revision mechanics.

Before the generator

Required Files halt early when missing. Include Files inject context. Checks Files record optional file presence. Deterministic Validation runs a Python command after generation.

Generator and judge pair

Generation Prompt does the work. Validation Prompt decides whether the result passes. Retry Prompt customizes revision instructions when the judge asks for changes.

Variant fork points

A prompt can fork into named variants. The selector path chooses a passing branch, and variant-sequential can trade speed for lower simultaneous quota use.

Optimization mode

Optimize runs a baseline, compares candidate model or effort settings, requires a passing winner, and emits an optimized prompt file.

The important boundary: prompt-runner executes prompt-defined control flow, but it does not know the methodology domain.

One Request, Eight Phases

Methodology-runner owns the end-to-end control flow for one change. Each phase points to a checked-in prompt module.

PH-000

Requirements Inventory

Extract traced RI items from the raw request.

PH-001

Feature Specification

Turn requirements into features and acceptance criteria.

PH-002

Architecture

Define components, technology choices, and integration points.

PH-003

Solution Design

Map features to concrete files, components, and interactions.

PH-004

Interface Contracts

Specify operations, data shapes, and behavior boundaries.

PH-005

Intelligent Simulations

Plan simulation artifacts when the architecture needs them.

PH-006

Incremental Implementation

Build the change with tests and implementation evidence.

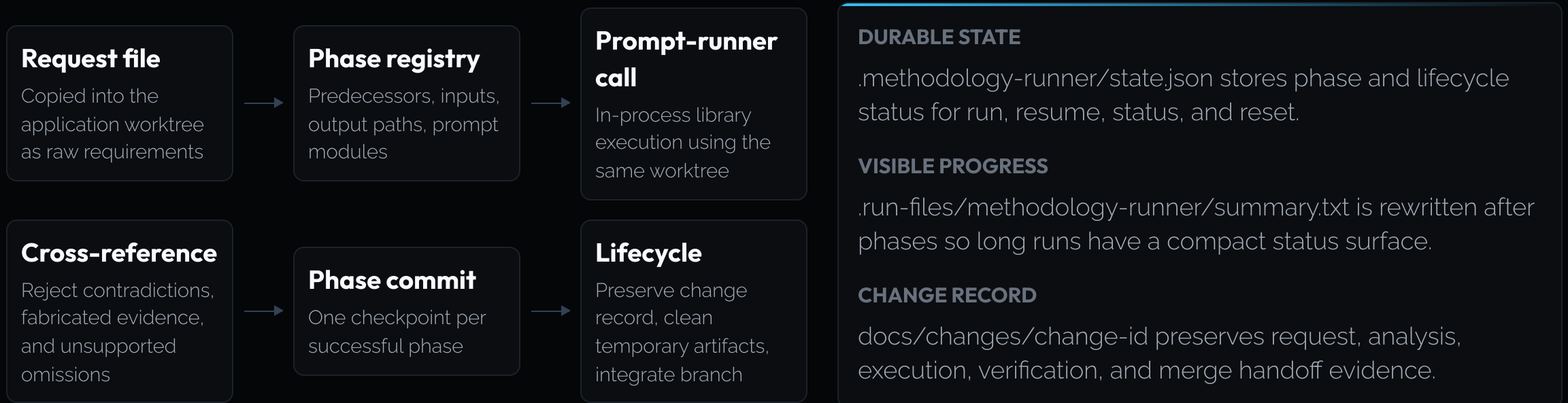
PH-007

Verification Sweep

Verify finished behavior against requirements and features.

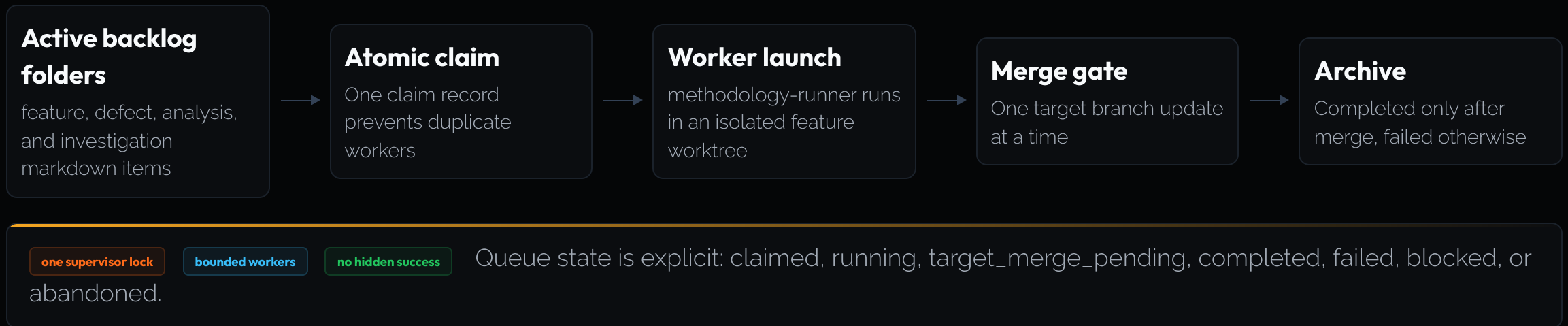
What It Adds Above Prompt-Runner

Prompt-runner executes one prompt module. Methodology-runner decides which phase runs, what it may read, what it must produce, and when the change can advance.



Queue Throughput Without Duplicating Methodology Logic

Backlog-runner supervises many methodology-runner executions. It does not write phase artifacts or call prompt-runner directly.



Parallel Work, Serialized Delivery

The split is deliberate: feature-branch work can run in parallel, but the target branch is a shared resource.



Final merge gate

Workers call methodology-runner with `--skip-target-merge`.
A successful worker writes `target-merge-handoff.json` and stops at `target_merge_pending`.

Backlog-runner then validates the handoff and merges one finalized source commit into the target branch at a time.

What Each Runner Owns

	prompt-runner	methodology-runner	backlog-runner
Unit	One prompt module or selected prompt pair.	One change request in one application worktree.	Many backlog items across a BacklogRoot.
Input	Markdown prompt file plus runtime placeholders.	Requirements markdown plus phase registry.	Typed backlog folders and application repo.
Quality Gate	Judge verdict and deterministic validation.	Phase validators plus cross-reference checks.	Worker result, merge handoff, and merge result.
State	Run directory and .run-files traces.	.methodology-runner state and phase summaries.	.backlog-runner claims, logs, state, and results.
Terminal Truth	Prompt pipeline passed, halted, or escalated.	Feature branch finalized or merged for one change.	Backlog item archived only after terminal outcome.

NORMAL COMMANDS

How People Touch The Stack

Validate or run a prompt module

```
$ prompt-runner parse  
tools/methodology-  
runner/docs/prompts/PR-025-ph000-  
requirements-inventory.md  
$ prompt-runner run prompt.md --  
backend codex --run-dir  
work/prompt-test
```

Run one methodology change

```
$ methodology-runner run request.md  
--workspace work/change --backend  
codex  
$ methodology-runner status  
work/change
```

Run backlog items

```
$ backlog-runner once --  
application-repo . --backend codex  
--max-workers 2  
$ backlog-runner status --backlog-  
root .
```

Use parse for prompt authoring, run or resume for a single change, and once or run for queue supervision.

Long Runs Need Evidence

The stack is designed so a stalled or failed run can be inspected without guessing from chat history.



Prompt traces

Every iteration can preserve rendered generator prompts, judge prompts, backend outputs, validation logs, and module summaries.



Methodology state

Status, resume, and reset operate from persisted phase and lifecycle state, not from memory of what happened earlier.



Queue truth

Claims, worker results, merge results, logs, and archive folders separate active, blocked, failed, merge-ready, and completed work.

SAFE RECOVERY PATTERN

Inspect status first, resume from saved state when possible, reset only the narrow phase or item boundary that actually failed.

ARCHIVE MEANING

Completed backlog is delivery evidence. Failed backlog is repair evidence. Active backlog stays dispatchable or blocked.

WHICH RUNNER SHOULD I REACH FOR?

Choose By Unit Of Work

prompt-runner

Use it when the problem is prompt authoring, prompt validation, model choice, or one reusable prompt workflow.

methodology-runner

Use it when one request needs the full requirements, design, implementation, and verification discipline.

backlog-runner

Use it when many backlog items need queue supervision, worker isolation, and serialized target-branch delivery.